Parallelizing Blockchain Computations

Andrew Photinakis
Rochester Institute of Technology
acp7795@rit.edu
Rochester, New York, USA

Caleb Talbott
Rochester Institute of Technology
cjt7922@rit.edu
Rochester, New York, USA

I. Introduction

The Blockchain is fundamentally defined as a distributed ledger or database composed of a sequence of blocks built upon a peer-to-peer (P2P) network [1], [2]. Its core essence is decentralization, meaning the system doesn't rely on central processing nodes. Instead, data is jointly maintained by nodes across the entire network, ensuring that rights and obligations are equal amongst participants [3].

A. Characteristics of Blockchain

The **Distributed Structure** of the Blockchain acts as a distributed data structure for recording transactions without a central authority [4]. **Consensus** is the protocol that enables participating nodes to append blocks to the chain in a unique, agreed-upon order [1]. The **Immutability and Security** of the Blockchain functions as an incorruptible and tamper-resistant ledger, which utilizes cryptographic verification to ensure data is not altered [2], [3]. **Transparency and Fault Tolerance** offer features such as provenance and resilience against node failures, allowing for the deployment of distributed applications in various settings [4].

B. The Scalability Trilemma

The "Scalability Trilemma," often referred to as the "impossible triangle," describes the inherent difficulty in blockchain systems to simultaneously achieve three conflicting objectives: **Decentralization, Security, and Scalability** [3], [2].

1) Decentralization: Decentralization ensures that the system does not rely on a central authority by distributing resources, such as computing power and stakes, to avoid corruption and collusion [3], [1]. A decentralized system is highly fault-tolerant and resistant to attacks because the failure of a single node does not stop the entire network [3].

2) Security: Security refers to ensuring data integrity through complete replication on each node and using cryptography for verification [3]. It implies two key properties: persistence (consistency of the ledger across honest nodes) and liveness (guaranteeing that transactions are eventually confirmed) [1]. Security mechanisms are necessary to defend against malicious behaviors, such as Sybil or double-spending attacks [1].

3) Scalability and Sequential Bottlenecks: Scalability refers to the system's efficiency as the workload increases, primarily measured by transaction throughput and confirmation latency [1], [3]. Traditional architectures, such as Proof-of-Work (PoW) and standard Practical Byzantine Fault Tolerance (PBFT), typically require every node in the network to process and validate every transaction to ensure global consistency [2]. This results in a sequential execution model that creates significant bottlenecks [1]. For instance, standard PBFT incurs a communication complexity of $O(N^2)$, which degrades performance rapidly as the number of nodes (N) increases [3], [5].

C. Project: Parallelizing Blockchain Computations

The goal of our project is to address these scalability bottlenecks by implementing a parallelized blockchain simulation. We adopt **Sharding** as a solution to scale the system horizontally. Sharding partitions the global validator set into smaller, independent groups known as "committees" or "shards," effectively dividing the transaction workload into disjoint sets [2], [4]. By enabling parallel transaction processing, the system's total throughput can theoretically increase linearly with the number of shards.

1) System Architecture: Our project implements a hierarchical structure to manage this parallelized workload while maintaining a unified ledger:

- **Root Process:** A designated process generates and distributes transactions to the leaders of individual shards.
- **Shard Committees:** The network is partitioned into N shards. Inside each shard, we utilize **Practical Byzantine Fault Tolerance (PBFT)** to achieve consensus on blocks of transactions. PBFT was selected for its ability to guarantee safety and liveness in systems where nodes may fail or behave maliciously (Byzantine faults) [5].
- **Final Committee:** To resolve the lack of a global view in individual shards, a "Final Committee" acts as an aggregator. It receives validated blocks from all Shard Leaders and sequences them into a final, globally ordered blockchain [3].

2) Implementation Scope: The simulation is implemented using **C++** and the **Message Passing Interface (MPI)** to model the distributed network, where each MPI process represents a single blockchain node. We utilize MPI_Comm_Split to divide the global pool of nodes into isolated communicators for

each shard and the final committee. The project targets a permissioned network setting under weak synchrony assumptions, where node identities are known and assigned specific roles (Shard Member, Final Committee Member) at initialization [5].

II. Outline of Algorithms and Programs

A. Algorithms/Methods

1) Network Partitioning (Sharding Logic): To utilize resources efficiently in systems with numerous nodes, the network can be partitioned into clusters. As described in the literature on sharding permissioned blockchains, nodes are partitioned into clusters where each cluster is responsible for processing specific transactions [4]. This approach allows independent transactions to be ordered and executed in parallel across different partitions [4].

The project implements this clustering concept in `src/main.cpp`. The `determineNodeAssignment` function calculates the role of a node based on its global MPI rank. This reserves the first set of ranks for the "Final Committee" and distributes the remaining ranks among the shards. The first rank in each shard partition is designated as the `SHARD_LEADER`. The code utilizes `MPI_Comm_split` to physically partition the global communication space `MPI_COMM_WORLD` into isolated sub-communicators (`shard_comm`). This creates distinct groups where nodes can communicate solely with their peers in the same shard or committee, effectively realizing the parallel clusters described by Amiri et al. [4].

2) Intra-Shard Consensus (PBFT): **Concept from Literature:** To establish consensus within a cluster where nodes do not trust each other, a Byzantine fault-tolerant protocol is required. The Practical Byzantine Fault Tolerance (PBFT) algorithm guarantees safety and liveness using a three-phase protocol: *pre-prepare*, *prepare*, and *commit* [5]. A request is considered committed when a replica has received a quorum of $2f + 1$ matching votes [5].

Implementation in the Project: The project implements this logic in `src/consensus/pbft.cpp`:

- **Pre-Prepare:** In the `prePrepare` method, the Shard Leader broadcasts the proposed `MicroBlock` (and its serialized data) to all replicas in the shard.
- **Prepare:** In the `prepare` method, replicas broadcast a `PREPARE` message containing the block hash after verifying the proposal. The `run` loop waits to collect a quorum of these messages.
- **Commit:** In the `commit` method, nodes broadcast a `COMMIT` message. The protocol finalizes the block once a node collects $2f + 1$ matching commit messages, aligning with the standard PBFT state machine [5].

Parallel Optimization: Within `src/consensus/pbft.cpp`, the `run` function includes a specific optimization for the verification step. The code uses OpenMP (`#pragma omp parallel for`) to

parallelize the loop that iterates through the transactions in a block proposal. This allows the cryptographic verification (`verify()`) of multiple transactions to occur concurrently on a single node, reducing the computational latency required to validate a batch.

3) Global Aggregation: **Concept from Literature:** In hierarchical or sharded consensus systems, a global consensus or proxy group is often responsible for merging blocks from different shards to form a global block [3]. This ensures that the data stored by nodes is comprehensive and maintains the global order of the ledger [3].

Implementation in the Project: The project implements this aggregation in `src/network/committee/final_committee.cpp`:

- **Collection:** The `collectMicroBlocks` method listens for incoming data from the specific global ranks of Shard Leaders. It gathers the verified `MicroBlock` objects produced by the parallel shards.
- **Assembly:** The `assembleMacroBlock` method takes the vector of collected microblocks and flattens their transactions into a single `MacroBlock`.
- **Storage:** In `src/main.cpp`, this new `MacroBlock` is appended to the global chain using `blockchain->addBlock`, creating a canonical history from the distributed work.

4) Cryptographic Primitives: **Concept from Literature:** Blockchain security relies on cryptographic verification to ensure data integrity and authenticity [2]. This typically involves digital signatures and cryptographic hashing to prevent tampering and prove ownership [3].

Implementation in the Project: The cryptographic operations are encapsulated in `src/util/crypto.cpp`:

- **Hashing:** The `keccak256` function implements the Keccak-256 algorithm (SHA3-256) using the OpenSSL EVP library (`<openssl/evp.h>`).
- **Signatures:** The `sign` and `verify` functions utilize the `secp256k1` library. This library provides Elliptic Curve Digital Signature Algorithm (ECDSA) functionality, allowing nodes to generate digital signatures for transaction data and verify them against public keys.

B. Block Diagrams/State Diagrams describing your solution

1) Figure 1: System Topology: **Theoretical Concept** is that the system topology visualizes a hierarchical sharding architecture designed to enhance scalability.

Clustering, as established in sharding literature, the network is partitioned into distinct clusters or shards. Each cluster is responsible for processing a disjoint subset of transactions, allowing for parallel execution rather than sequential validation [4].

Hierarchical Aggregation To ensure a unified global ledger state, the architecture employs a hierarchical approach where a "global proxy node" or committee is responsible for merging the blocks produced by individual shards [3]. This ensures that the data stored across the network is comprehensive and consistent [3].

Project Implementation: Figure 1 illustrates the data flow and node hierarchy implemented within `src/main.cpp`.

- **Root Process:** The root of the tree (MPI Rank 0) acts as the transaction generator and distributor. In `src/main.cpp`, the main function initializes the transaction pool and partitions it, sending specific subsets to Shard Leaders using `MPI_Send`.
- **Shard Leaders:** The intermediate nodes represent Shard Leaders. Upon receiving transactions, they coordinate the intra-shard consensus to generate `MicroBlock`s.
- **Final Committee:** The topology converges at the Final Committee. Implemented in `src/network/committee/f` the `collectMicroBlocks` function aggregates the verified blocks from Shard Leaders, and `assembleMacroBlock` flattens them into the canonical blockchain.

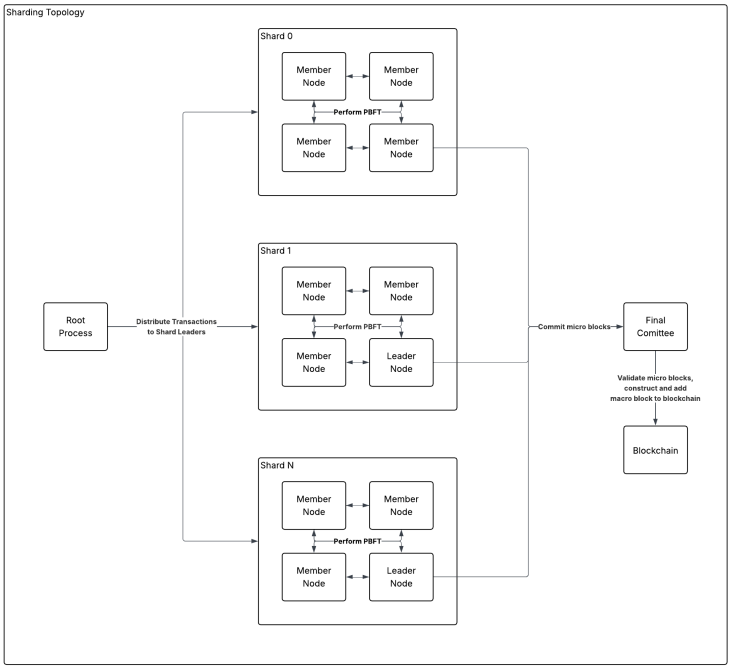


Fig. 1. System Topology: Visualizing the hierarchical flow from the Root transaction distributor, through parallel Shard Leaders, to the Final Committee aggregator.

2) Figure 2: PBFT State Machine: The Practical Byzantine Fault Tolerance (PBFT) algorithm operates as a finite state machine to guarantee safety and liveness in the presence of Byzantine faults.

- **State Transitions:** The protocol progresses through a specific sequence of states:
  - **Pre-Prepared:** A replica enters this state upon receiving a valid proposal and a pre-prepare message from the primary [5].
  - **Prepared:** A replica transitions to the prepared state once it has collected a quorum ($2f$) of valid prepare messages from other replicas, ensuring that non-faulty nodes agree on a total order [5].
  - **Committed:** The final state is reached when a replica collects a quorum ($2f + 1$) of commit messages, at which point the transaction is finalized and executed [5].

Project Implementation: Figure 2 maps directly to the control logic within the `PBFT::run` method in `src/consensus/pbft.cpp`.

- `NEW` $\rightarrow$ `PRE-PREPARED`: The state machine begins when the leader executes `prePrepare`, broadcasting the block. Replicas verify the proposal and enter the protocol.
- `PRE-PREPARED` $\rightarrow$ `PREPARED`: Replicas broadcast `PREPARE` messages. The code executes a `while` loop, waiting to accumulate a quorum of these messages. Successful exit from this loop signifies the transition to the Prepared state.
- `PREPARED` $\rightarrow$ `COMMITTED`: Upon entering the Prepared state, the node broadcasts a `COMMIT` message. It then enters a final `while` loop, waiting for a quorum of commit messages. Exiting this loop marks the transition to the Committed state, returning the agreed-upon block.

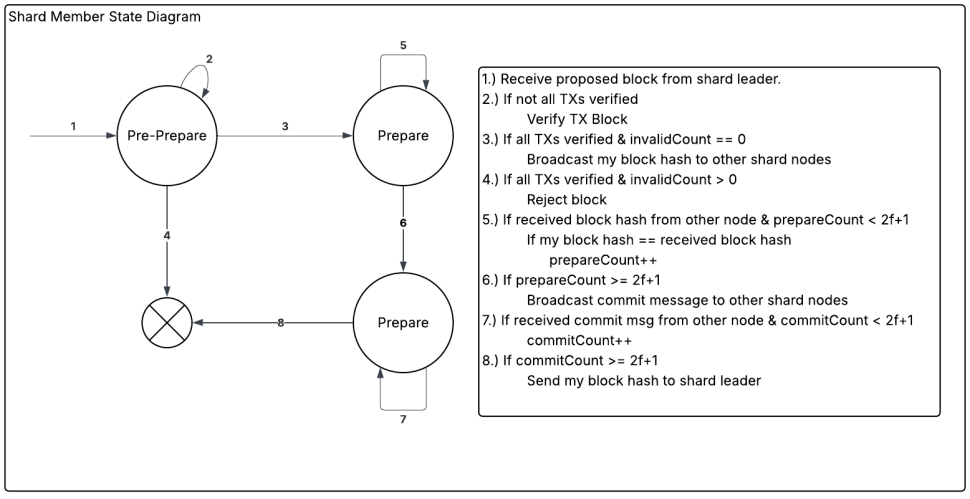


Fig. 2. PBFT State Machine: Illustrating the transitions (NEW $\rightarrow$ PRE-PREPARED $\rightarrow$ PREPARED $\rightarrow$ COMMITTED) enforced by the consensus logic in `src/consensus/pbft.cpp`.

III. Results

- **Experimental Setup:**
  - **Hardware:** RIT `kraken` cluster (Debian 12, Xeon CPU).
  - **Parameters:** Experiments run via `simulation.py`, varying shard counts ($S \in \{1, 4, 8, 16\}$).
- **Speed Analysis:**

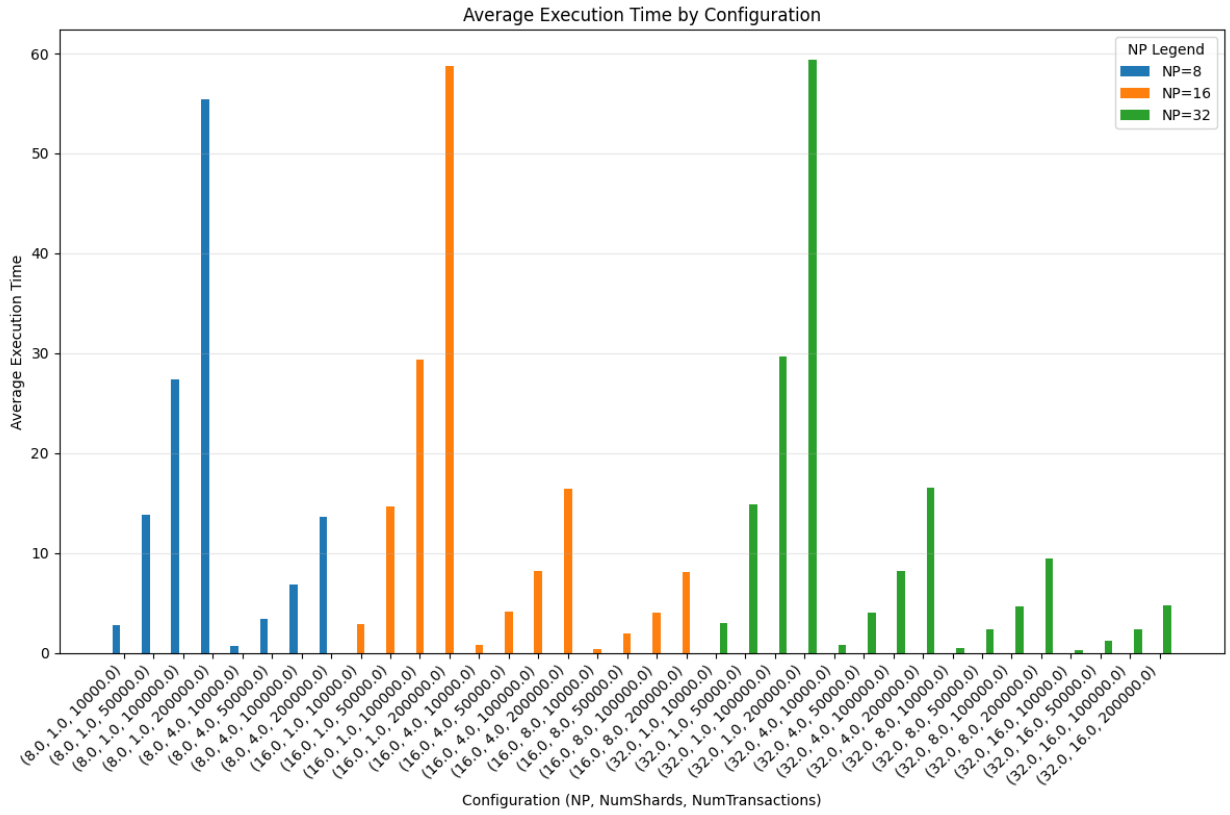
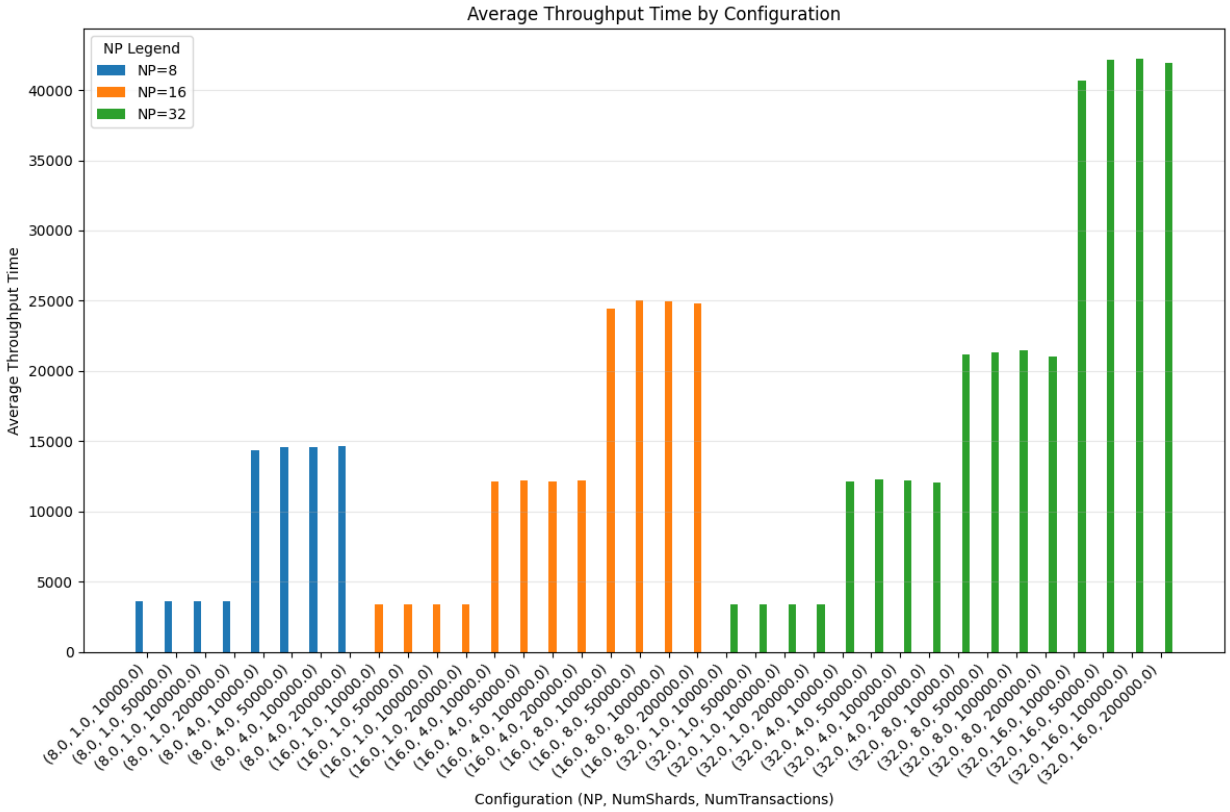


Fig. 3. Graph showcasing excecution time for various runs on $P \in \{8, 16, 32\}$ and $S \in \{1, 4, 8, 16\}$

- **Finding:** Execution times decrease exponentially with the number of shards. Additionally, introducing more processors slightly decreases execution times.

- **Throughput Analysis:**



– **Findings:** Throughput increases linearly with the number of shards. By partitioning the transaction pool (‘partitioned_txs‘ in `main.cpp`), we utilize more resources to process a larger total workload, validating Gustafson’s Law.

IV. Documented Code

A. *Codebase Structure*

The project’s source code is organized into a modular structure under the `src/` directory, reflecting the architectural separation between state maintenance, consensus logic, and network topology required in sharded systems [4].

- **`src/core/`:** This directory contains the definitions for the project’s fundamental data structures. It includes classes for the `Blockchain` itself (`blockchain.cpp`), the `Node` (`node.cpp`), `Transaction` (`state/transaction.cpp`), and the various block types like `Block` (`blocks/block.cpp`), `MicroBlock` (`blocks/micro_block.cpp`), and `MacroBlock` (`blocks/macro_block.cpp`). These components represent the immutable ledger state exchanged throughout the simulation [1].
- **`src/consensus/`:** This directory houses the logic for achieving agreement among nodes. Its primary component is the implementation of the Practical Byzantine Fault Tolerance algorithm in

pbft.cpp. This code manages the multi-phase commit process (Pre-Prepare, Prepare, Commit) required for a shard to validate a MicroBlock [5].

- **src/network/:** This directory is responsible for managing communication between simulated nodes. It contains high-level abstractions like Shard (shard.cpp) and FinalCommittee (committee/final that encapsulate the group behaviors of the hierarchical topology [3]. Critically, it also includes mpi_wrapper.cpp, which provides simplified send and recv functions built on top of MPI for exchanging serialized data across the network.

B. Documentation

The codebase is documented using Doxygen-style comments to ensure the clarity of the consensus state machine and network logic. As seen in files like src/network/shard.cpp and src/consensus/pbft.cpp, this convention is applied at multiple levels:

- **File-level headers** (@file, @brief) describe the purpose of the entire file.
- **Class comments** explain the role and responsibilities of a class (e.g., Shard, PBFT).
- **Method comments** (@brief, @param, @return) detail the purpose, inputs, and outputs of individual functions, such as the broadcastMessage function in the PBFT implementation.

C. Key Implementation Details

To transmit complex C++ objects (like Transaction or MicroBlock) over MPI, which operates on raw byte streams, the project uses a custom serialization system defined in src/util/serialization.cpp. This addresses the data handling requirements inherent to high-performance computing environments [6].

- The pack() functions convert primitive types (e.g., int, double, std::string) into a sequence of bytes and append them to a std::vector<char> buffer.
- The unpack_*() functions read from this byte buffer at a given offset, reconstruct the original data type, and advance the offset.

This byte-level serialization is essential for converting the project's C++ data structures into a portable format that can be sent and received by the network wrappers, which call the underlying MPI communication routines.

V. Manual

A. Build Instructions

The project is compiled using the make utility, as defined in the Makefile found in the root directory.

Command: Running `make` from the project root will compile the source code and link the final executable.

```
make
```

This generates the main binary at `build/main`.

Dependencies: The build process relies on several key libraries to support the simulation's requirements:

- **C++17 Compiler:** Required for modern language features, indicated by the `-std=c++17` flag in the build configuration.
- **OpenMPI:** The `mpic++` compiler wrapper is used for compilation and linking to support the distributed node simulation [6].
- **OpenSSL:** Linked via the `-lcrypto` flag, providing the cryptographic functions (SHA-256, ECDSA) used in `src/util/crypto.cpp` to ensure data integrity [2].
- **OpenMP:** The `-fopenmp` flag is used to enable multi-threaded parallelism for tasks like parallel signature verification within the PBFT protocol.

B. Execution Guide

The simulation is executed via the `mpirun` command, which launches the distributed processes.

Command Structure:

```
mpirun -np <number_of_processes> build/main [ARGUMENTS]
```

Example Execution: The following command runs the simulation on 8 processes, configured to create 6 shards. It injects 100,000 transactions and sets other parameters for the run.

```
mpirun -np 8 build/main --shards 6 --transactions 100000 --run-id
1 --seed 42 --transaction-size 256 --faults 0.2 -v 1
```

Key Arguments:

- `--shards <N>`: Divides the processes into N shards, allowing the evaluation of scalability [3].
- `--transactions <N>`: Sets the total number of transactions to be generated and processed.
- `--faults <F>`: Specifies the fraction of faulty transactions to inject, testing the robustness of the consensus [5].
- `-v <level>`: Sets the verbosity level for logging.

VI. Conclusions

A. Summary

The project successfully simulated a parallelized blockchain architecture that circumvents the sequential processing limitations inherent to traditional ledgers [2]. By partitioning the network into independent shards the system demonstrated the capability to process disjoint transaction sets concurrently. This architecture, orchestrated by the logic in `src/main.cpp`, validates that horizontal scaling can be achieved by decoupling the consensus workload across multiple intra-shard PBFT instances [5].

B. Critique & Challenges

- **Serialization Overhead:** A significant performance challenge identified during the implementation was the CPU cost associated with data transmission. MPI communication requires data to be transmitted as raw byte streams. The project utilizes custom byte-packing logic implemented in `src/util/serialization.cpp` to convert complex C++ objects (like `Transaction` and `Block`) into portable formats. The manual packing and unpacking introduce a non-trivial computational overhead that acts as a serial component in the parallel workflow.
- **Aggregation Bottleneck:** The system architecture relies on a central `FinalCommittee` to maintain the global ledger, as implemented in `src/network/committee/final_committee.cpp`. While sharding parallelizes verification, the aggregation step remains centralized; the committee must sequentially receive and order blocks from every shard leader. This finding aligns with sharding literature, which identifies global root chains as potential bottlenecks that shift the scalability limit rather than eliminating it entirely [2], [3].
- **Fault Tolerance and Dynamic Recovery:** The simulation operates under a ”fault aware” design but lacks true dynamic recovery. Hursey et al. describe MPI implementations that can sustain performance across process failures through run-through stabilization [6]. Currently, the simulation relies on `MPI_Abort` if a critical failure occurs (e.g., leader crash), terminating the entire workload. While the PBFT logic handles Byzantine faults within a shard [5], the system cannot yet dynamically recover from node crashes without restarting.

C. Future Work

- **Cross-Shard Transactions:** The current simulation assumes transactions are contained within single shards. Future iterations must implement atomic cross-shard protocols. The codebase includes `src/network/cross_shard.cpp` as a structural stub for this functionality. Supporting this

would require complex coordination mechanisms, such as two-phase commit, to ensure that a transaction touching multiple shards is committed atomically [4].

- **Decentralized Final Committee:** To address the aggregation bottleneck, the Final Committee itself should be decentralized. Instead of a single leader aggregating blocks, the committee should run its own instance of a BFT consensus protocol to agree on the ordering of shard blocks. This would remove the single point of failure and align the architecture with fully decentralized sharding models [2].

REFERENCES

[1] J. Xu, C. Wang, and X. Jia, "A survey of blockchain consensus protocols," *ACM Comput. Surv.*, vol. 55, no. 13s, Jul. 2023. [Online]. Available: https://doi.org/10.1145/3579845

[2] G. Yu, X. Wang, K. Yu, W. Ni, J. A. Zhang, and R. P. Liu, "Survey: Sharding in blockchains," *IEEE Access*, vol. 8, pp. 14 155–14 181, 2020.

[3] X. Wu, W. Jiang, M. Song, Z. Jia, and J. Qin, "An efficient sharding consensus algorithm for consortium chains," *Scientific Reports*, vol. 13, no. 1, p. 20, 2023.

[4] M. J. Amiri, D. Agrawal, and A. El Abbadi, "On sharding permissioned blockchains," in *2019 IEEE International Conference on Blockchain (Blockchain)*, 2019, pp. 282–285.

[5] M. Castro and B. Liskov, "Practical byzantine fault tolerance," in *3rd Symposium on Operating Systems Design and Implementation (OSDI 99)*. New Orleans, LA: USENIX Association, Feb. 1999. [Online]. Available: https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance

[6] J. Hursey and R. L. Graham, "Analyzing fault aware collective performance in a process fault tolerant mpi," *Parallel Computing*, vol. 38, no. 1, pp. 15–25, 2012, extensions for Next-Generation Parallel Programming Models. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167819111001414